* For i = 0 , j = 1.

int min Index = 0

~~for~~ arr[j] < arr [minIndex] { 22 < 18 ×

minIndex = j;

}

// min Index = 0

* For i = 0 ; j = 2

arr[j] < arr[minIndex] { 20 < 18 ×

~~min Index~~ minIndex = j

}.

// minIndex = 0

* For i = 0 ; j = 3

arr[j] < arr[minIndex] { 19 < 18 ×

minIndex = j

}

// minIndex = 0

}

* For i = 0 ; j = 4

arr[j] < arr[minIndex] { 21 < 18 ×

minIndex = j

}

minIndex = 0

}

int temp = 18

arr[0] = arr[0]

arr[0] = 18

Similar for remaing loops.

Q-2 {34, 12, 56, 45, 23}

min

34	12	56	45	23

min

12	34	56	45	23

12	23	56	45	34

12	23	34	45	56

Bubble Sort :-

30	25	27	35	29

25	30	27	35	29

25	27	30	35	29

25	27	30	29	35

25	27	29	30	35

Code:

```
int n = arr.length.

for (int i=0 ; i<n-1 ; i++){
  for (int j=0; j<n-i-1; j++){
      if (arr[j] > arr[j+1]){
          int temp = arr[j]
          arr[j] = arr[j+1];
          arr[j+1] = temp
      }
  }
}
```

dry run:

For i = 0 , j = 0

if (arr[0] > arr[1]){ 30 > 25 ✓

int temp = 30

arr[0] = 25

arr[j+1]/arr[1] = 30

25	30	27	35	29
0	1	2	3	4

★ For i = 0 ; j = 1

if (arr[1] > arr[2]) ✓ {

int temp = 30

arr[1] / arr[j] = 27

[arr[2] / arr[j+1] = 30

25	27	30	35	29

★ For i = 0 ; j = 2.

if (arr[2] > arr[3] 30 > 35 ✗ {

}

★ For i = 0 ; j = 3

if (arr[3] > arr[4] ✓ { 35 > 29

int temp = 35

arr[3] = 29

arr[4] = temp.

25	27	30	29	35
0	1	2	3	4	0	1	2	3	

★ For i = 1 ; j = 0

if [arr[0] > arr[1] ✗

★ For i = 1 ; j = 1.

if (arr[1] > arr[2] ✓ ✗

★ For i = 1 , j = 2

if (arr[2] > arr[3]) { 30 > 29 ✓

15, 10, 25, 40, 30

15	10	25	40	30
0	1	2	3	4

10	15	25	40	30

10	15	25	30	40

```
* For (int i=0; i<n-1; i++){
    boolean swapped = false;
    For (int j=0; j<n-i-1; j++){
        if (arr[j] > arr[j+1]){
            int temp = arr[j];
            arr[j] = arr[j+1]
            arr[j+1] = temp
            swapped = true
        }
    }
    if (!swapped){
        break;
    }
```

dry run:

For i=0; j=0

swapped = false

if (arr[j] > arr[j+1]) 15 > 10 ✓ {

int temp = 15

arr[j] = 10

arr[j+1] = 15

swapped = true

10	15	25	40	30

* For i=0 ; j=1.
  swapped = False
  if (arr[1] > arr[2]) { 15 > 25 ✗
  }
  swapped = false

* For i=0 ; j=2
  swapped = False
  if (arr[2] > arr[3]) { 25 > 40 ✗
  }
  swapped = true

* For i=0 ; j=3.
  arr[3] > arr[4] { 40 > 30 ✓
  int temp = 40
  arr[3] = 30
  arr[4] = temp
  swapped = true

10	15	25	30	40

* For i=1 ; j=0
  arr[0] > arr[1] { 10 > 15 ✗
  — swapped = false
  }

* For i=1 ; j=1
  arr[1] > arr[2] { 15 > 25 } ✗
  — swapped false

Q3. Insertion Sort:

18	22	20	19	21

18	20	22	19	21

18	20	19	22	21

18	19	20	21	22

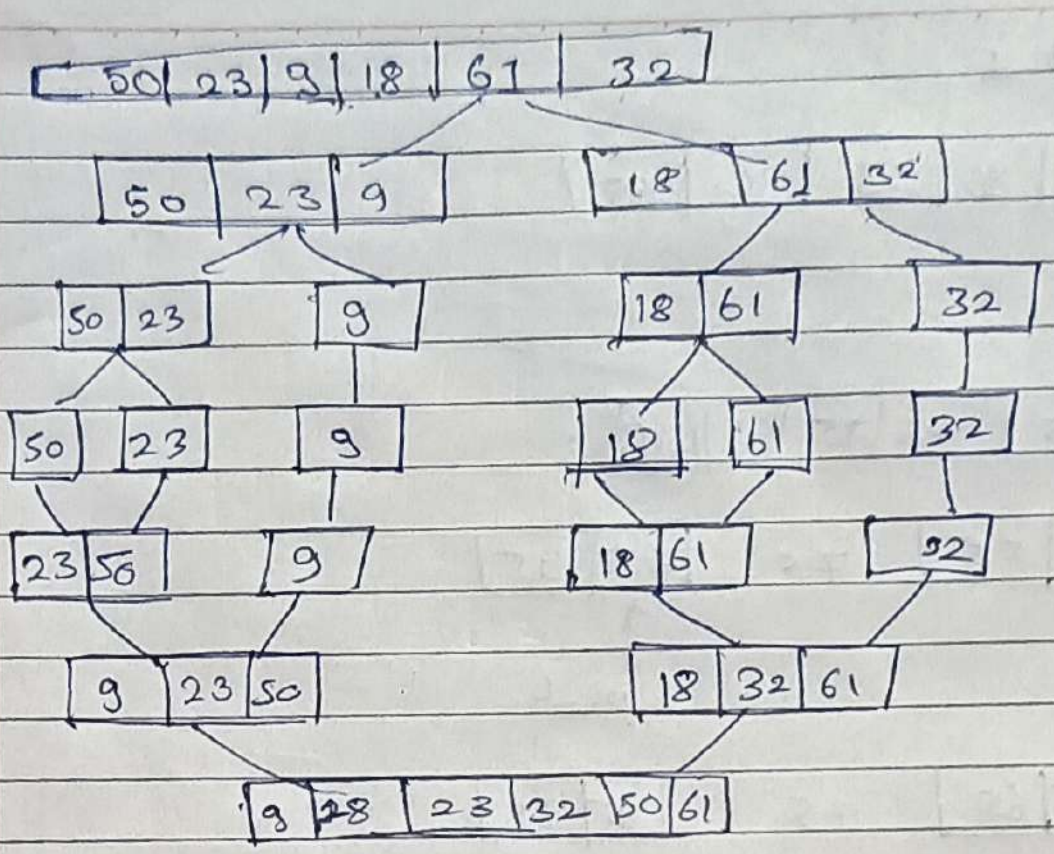
```
For (int i=1 ; i<n ; i++){
   int CurrentElement = arr [i]
   int left = i-1 ;
    while (left ≥ 0 & arr [left] > current Element){
        arr[left + 1] = arr [left];
            left --;
        }
        arr [left +1] = current element.
```

dry run

* i = 1 ;

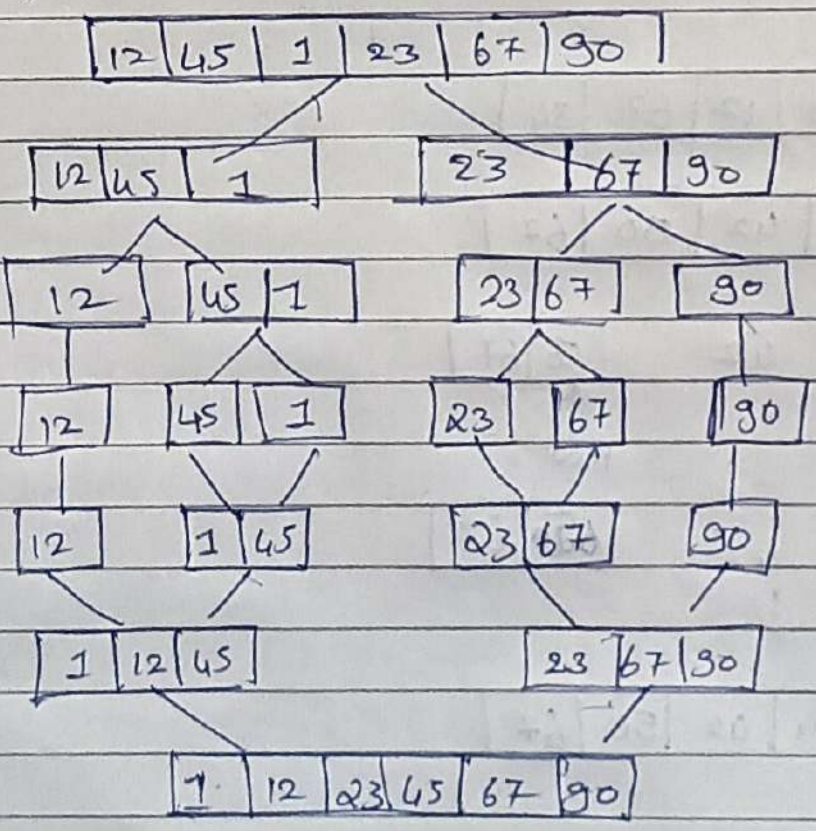
```
int CurrentElement = arr[1]
int left = 1-1 = 0
  while (left ≥ 0 & arr [left] > current element){
      arr [0+1] = arr [left] ; arr[0]
          left -- ; -1
      arr [left +1] 0
```

1.)


50 23 9 18 61 32
50 23 9
18 61 32
50 23
9
18 61
32
50
23
9
18
61
32
23 50
9
18 61
32
9 23 50
18 32 61
9 18 23 32 50 61

Code:

~~Bonus~~


12 45 1 23 67 90
12 45 1
23 67 90
12
45 1
23 67
90
12
45
1
23
67
90
12
1 45
23 67
90
1 12 45
23 67 90
1 12 23 45 67 90

Input 1 :

75	85	55	95	65

↑ Pivot

65	55	75	85	95

↓

65	55	75	85	95

↑ Pivot ↑ Pivot

55	65	75	85	95

→

55	65	75	85	95

Input 2 :

↓ Pivot

42	50	12	67	34

34	12	42	50	67

34	12	42	50	67

↑ Pivot ↑ Pivot

12	34	42	50	67

↓

12	34	42	50	67